ENGINEERING AUDIT / 2026

Placement Co-pilot

Codebase audit, implementation assessment, risk analysis and delivery roadmap

AUDIT DATE	REPOSITORY	BRANCH / COMMIT
29 October 2026	E:\placement-copliot	main / f7387689df78
PROJECT	STACK	SCOPE
Placement Co-pilot	Next.js 16.4.2, React 19, TypeScript 5, Supabase, FastAPI	Repository source, docs, migration, configs

Overall assessment: substantial interactive prototype with meaningful backend groundwork; core end-to-end behavior remains unverified and persistence/authentication gaps remain.

Prepared from the currently open repository only. No application source, database, deployment or Git history was changed. The existing user modification to .env.example was preserved; only variable names were inspected.

CONFIDENCE: MEDIUM-LOW

The repository contains detailed implementation documents, but no formal product requirements, automated test suite, configured external-service credentials, or successful end-to-end test evidence. Scores are therefore provisional and tied to the scope evidenced by the repository.

1. Executive summary

The repository is a student placement assistant called Placement Co-pilot. It combines a polished multi-route UI, client-side services, Next.js route handlers, optional Supabase persistence, optional Gemini/Groq calls, external job feeds and a companion FastAPI document parser. Its implementation is materially beyond a static mockup: application tracking, readiness updates, interview evaluation and resume parsing have real code paths. However, the project's own claims of production readiness and complete verification are not supported by the current checkout.

Weighted provisional implementation estimate: **70%**. Functional verification of major user journeys: **0%** in this audit. Static TypeScript checking and Python syntax checking passed; lint failed; production build could not complete because Google Fonts could not be fetched. No automated tests or in-repository test fixtures were found.

Highest priority findings are an unscoped process-wide job-match cache that ignores user skills and target role; public AI-backed route handlers without authentication, rate limits or explicit resume upload size limits; resume analyses returned to the browser but not persisted in live mode; and mismatch between documentation's claims and the current executed lint/build outcomes.

Scope basis

No standalone product requirements or acceptance criteria were found. Scope was provisionally derived from PROJECT_OVERVIEW.md, docs/FULL_PROJECT_IMPLEMENTATION.md, docs/SUPABASE_SETUP.md, route/page descriptions, schema and service behavior. This is an inference from project-authored material. Completion can change when requirements are clarified.

Important: documentation states 'Production-Ready' and claims clean lint/build results. This audit re-ran lint and build: lint reports 6 errors and 29 warnings; build stops while fetching the Inter font. Documentation's reported runs are historical claims, not verification of this checkout.

2. Repository identity, stack and architecture

Layer	Observed implementation	Current evidence / boundary
UI	Next.js App Router, React client pages, TypeScript, Tailwind CSS v4, lucide-react	Six main feature pages plus login; UI is mostly client-rendered. No browser journey run.
Server endpoints	Next.js Route Handlers under src/app/api	Resume analysis, interview evaluation, job search/matching, system status.
Services	src/lib/services; mode switch between demo and Supabase	Some reads/writes use Supabase; demo state is module memory. Jobs also use localStorage for skill preferences.
Database/auth	Supabase SSR/browser clients; one SQL migration with 11 tables and RLS	Schema is code only. No credentials, live DB, migration application or RLS exercise in this audit.
AI / job feeds	Gemini and Groq via server fetch; Adzuna then Remotive then local catalog	Credential-gated; fallback may be deterministic or mock. Integrations not exercised.
Parser	Optional Python FastAPI service with PyMuPDF, pypdf and python-docx; native Next fallback	Next route probes PYTHON_PARSER_URL, else attempts internal extraction. No parser server launched.
Verification	ESLint, TypeScript compiler, Next build scripts	No test script or test files. Only static checks were run.

High-level data flow

Student browser / route pages	Next.js Route Handlers	Supabase / AI providers / job APIs
forms, state, UI	resume, evaluate, jobs, status	Postgres + RLS; Gemini/Groq; Adzuna/Remotive
Service layer	Optional Python parser	Local demo data
mode-aware reads and writes	FastAPI /parse-resume	in-memory seed + localStorage

The request paths generally connect to service functions and server handlers. Database state is per-user where the services filter by user id and RLS policies are defined. Demo state is in module memory, so it is process/session-local and resets on refresh or redeploy. The project has no deployment configuration, CI workflow, or Supabase CLI config in the tracked repository.

Repository inventory: 62 tracked/non-generated project files as returned by rg --files, including 35 source files under src, one Python parser, one SQL migration, 3 substantive markdown documents, config/manifests and static assets. node_modules and .next were excluded from source analysis.

3. Intended scope and feature inventory

There is no separate product requirements document. The feature set below is a provisional interpretation of the project overview and implementation guide, not an externally validated product spec.

Feature / outcome	Evidence / main files	Expected flow / dependency	Classification
Account sign-in and profile	login page; auth.service; callback; proxy; profile schema	Supabase Auth -> session cookies -> profile query; Supabase required for live mode	C - Partial
Placement dashboard	dashboard page/service; mock data	queries to applications, sessions, resume analyses, readiness tasks	C - Partial
Resume parsing, scoring and tailoring	resume page; analyze route; resume.service; Python parser	Upload/text -> parse -> AI or heuristic analysis -> result and save/edit	C - Partial
Job discovery and matching	jobs page/service; search/match routes	filters -> Adzuna/Remotive/mock; Gemini or heuristic match	B - Implemented, not verified
Application tracking	tracker page; applications.service; applications schema	create/edit/move/delete -> Supabase or in-memory demo store	B - Implemented, not verified
Mock interview coach	interview page/service; evaluate route; sessions/turns tables	prompt -> browser transcript/telemetry -> AI/heuristic feedback -> save session	B - Implemented, not verified
Readiness plan	readiness page/service; task schema	read task list -> toggle completion -> persist	C - Partial
Companion document parser	python-parser/main.py; resume analyze route	optional HTTP call -> native fallback	B - Implemented, not verified

Classification definitions used: A verified; B code present but not behaviorally verified; C partial; D mock/placeholder only; E concrete failure; F required but absent; G out of scope. A passing type check is not end-to-end verification. No feature qualifies for A.

Data mode and persistence

The README remains the create-next-app starter and does not explain this product's behavior. More detailed docs claim demo/live dual mode and persistent records. Services confirm that applications, readiness toggles, interview sessions and some profile/analysis reads query Supabase in live mode. Demo state is held in module variables. Job search uses an API route that can call public/external feeds and then return curated mock results; it does not query Supabase in that route. Resume analysis does not write its newly generated output to resume_analyses, despite a live-mode read/update service and schema.

4. End-to-end flow tracing

Journey	Observed trace	Result and gap
Sign in	login page -> signInWithEmail/signUpWithEmail -> Supabase browser auth -> callback exchanges code -> proxy refreshes session	Auth calls are connected. Proxy only refreshes session and redirects authenticated users away from login. It does not enforce protected routes. No live credentials/test.
Analyze resume	resume page upload -> analyzeResumeFile -> POST /api/resume/analyze -> optional Python probe -> unpdf/mammoth/native parsing -> Gemini -> Groq -> deterministic scoring -> JSON -> page state	Useful real pipeline. No explicit file-size cap in Next handler, file type relies mainly on extension/MIME detection, no auth or rate limit, and result is not persisted in live mode. External engines untested.
Search and match	jobs page -> searchJobs /api/jobs/search -> Adzuna -> Remotive -> curated mock; match API -> Gemini/heuristic -> filters and card updates	Connected code and visible source labels. No external call verified. Match cache key omits userSkills and target role/job content, so same job ID/preferences can reuse stale cross-profile score.
Track application	jobs apply / tracker add -> applications.service -> insert in Supabase or mutate demoCards -> reload list	Live user filter and schema align. No DB configured; no integration test. Demo mutations are memory-only.
Practice interview	interview page -> evaluateInterviewAnswer -> Groq -> Gemini -> deterministic evaluator -> result UI -> recordInterviewSession -> session and turn inserts	Scoring fallback means AI-looking evaluation can be heuristic; source marker exists. Insert cleanup is best effort, not transactional. No AI/DB journey tested.
Readiness	readiness page -> getReadinessMilestones -> query readiness_tasks -> toggleMilestone update; demo updates module array	Existing rows can be toggled. Migration contains no default task seed; UI has no task-creation path. Live empty DB means no prepared checklist.
System status	Topbar fetch -> /api/system/status -> reports config flags and attempts parser /health	Useful diagnostics; route has no auth and exposes parser URL/configuration metadata. Reported configured status is not proof of successful DB/API connection.

Error handling is mixed. Some live services throw and pages show errors; searchJobs catches and silently returns demo results, while scoreJobMatches catches and leaves catalog scores. These can make an unavailable integration look like normal job results unless source metadata is surfaced consistently.

5. Confirmed defects and probable risks

Severity reflects potential user/system impact. Confidence is Confirmed when the code path itself establishes the issue; Strong evidence for a material risk whose exploitability depends on hosting/configuration; Possible risk for a plausible failure not reproduced.

ID / severity / confidence	Evidence and impact	Recommended fix / verification
R1 - High / Confirmed	Job match cache in src/app/api/jobs/match/route.ts:506-555 keys only by jobId and preferred location/types. It omits userSkills and target role, even though scoring consumes them. Same runtime can return one user's score to a different profile or stale changed job. Wrong relevance and potential cross-user inference.	Include normalized scoring inputs and job content/version in key, or scope cache by user; validate with two profiles and changed job under identical job ID.
R2 - High / Strong evidence	src/app/api/resume/analyze/route.ts:940-1044 and src/app/api/interview/evaluate/route.ts:507-540 accept requests without authentication/rate limit. Resume route buffers the complete multipart file at :960 and has no explicit size ceiling. It may invoke paid provider keys at :1016-1025. Hosting limits can constrain impact, but the app code has no guard.	Add server-side authentication/abuse controls where product scope requires them, file size/type limits before buffering/parsing, request schema bounds, provider quotas/timeouts. Verify unauthenticated rejection, oversized upload rejection and throttling.
R3 - High / Confirmed against documented live persistence	Resume analysis returns JSON at analyze/route.ts:1033 and analyzeResumeFile only assigns module-level demoAnalysis in resume.service.ts:97-108. No insert to resumes/resume_analyses occurs in the analyze flow. Live getResumeAnalysis reads the DB at resume.service.ts:34-82. New live analysis is therefore not available after reloading the page; saved analysis table remains unchanged.	Persist analysis and resume metadata for authenticated users, or clearly scope session-only behavior. Verify upload -> reload -> analysis present in database and UI.
R4 - Medium / Confirmed	src/proxy.ts:10-24 runs refresh, but redirects only signed-in users visiting /auth/login. No protected route check appears. Client service methods block some data queries, but pages and public handlers remain callable. Live users can reach pages unauthenticated and see failed/empty states; route protection is not enforced as docs suggest.	Define intended guest/live access and enforce it at server page/data boundaries and each protected route. Verify direct URL and API requests with anonymous and authenticated sessions.
R5 - Medium / Confirmed	The readiness migration creates tables/policies but seeds no per-user tasks. readiness service supports read/toggle only, and readiness UI has no create flow. A new live account can receive an empty readiness list, with fixed domain constants rather than live domain calculations.	Add explicit onboarding/seed behavior or task creation and define whether domain scores are static or data-derived. Verify a new account can populate and complete tasks.
R6 - Medium / Strong evidence	interview.service.ts:164-173 inserts a session then turns; on child failure it attempts parent delete but ignores deletion error. This is compensating cleanup, not atomicity. Partial session can remain if cleanup fails.	Use a transactional database function/RPC or document recoverable partial state; inject turn insert failure and inspect database.
R7 - Medium / Confirmed	npm run lint returned 6 errors and 29 warnings. Errors include explicit any in resume route :809, :820, :959 and React set-state-in-effect findings in interview page :225/:441 and jobs page :425. Current checkout cannot claim clean lint.	Resolve lint findings and warnings, then rerun npm run lint in CI.
R8 - Medium / Strong evidence	PROJECT_OVERVIEW.md and docs/FULL_PROJECT_IMPLEMENTATION.md state complete/production-ready and report clean lint/build. Current lint fails; build is blocked at next/font Google font network fetch. Documentation is stale or not repeatable in current environment, reducing trust in readiness claims.	Replace unsupported status language with dated reproducible evidence; make build independent of external font fetch or run in network-enabled CI and archive logs.
R9 - Low / Confirmed	src/app/api/system/status/route.ts reports configured services and parser URL/details without auth. It does not perform a Supabase query or AI provider call. Labels can be mistaken for verified connectivity, and parser endpoint metadata is exposed.	Return only necessary public health detail; distinguish configured from reachable/authenticated and verify response policy.
R10 - Low / Confirmed	No package test script, test directories, unit/integration/E2E tests, CI workflow or coverage config were found. Error handling and state behavior have no automated regression evidence.	Add focused tests around auth boundaries, persistence, cache key, parsing limits, scoring/fallback and key user journeys.

6. Technical quality, security and data integrity

Positive implementation evidence

- Supabase migration defines 11 tables, indexes, RLS enablement and user-specific policies. Child interview turns use a parent-session ownership predicate; the signup trigger sets a fixed public search_path.

- Live application/readiness/session/profile services generally check the signed-in user and filter by user id in client queries; database RLS remains the primary row boundary.

- Gemini/Groq requests are server-side and use AbortSignal timeouts in the relevant route handlers. Job search and analysis disclose source/fallback metadata in API responses.

- TypeScript strict mode is enabled and the full no-emit compiler check passes.

Areas needing verification

- No database instance or credentials were available; migration syntax, policy behavior, triggers, query mappings and live auth were not executed.

- Resume parsing is heuristic. Native PDF stream extraction covers a subset of PDF encodings; scanned-image OCR is absent, and file size is capped only in the optional Python service (5 MB), not in the Next handler.

- Interview fallback calculates deterministic scores from response content/telemetry. It is useful practice logic but not equivalent to AI evaluation; users should see the source clearly. No microphone/camera/browser permission flow was exercised.

- Demo writes to module state are volatile. Job skills use browser localStorage, while saved job status behavior was not proven to persist across reload or users.

- The status endpoint only checks key presence/configuration and parser health, not actual DB or model connectivity; avoid treating its green/configured state as a passed integration test.

Secrets review

Only environment-variable names and consuming code were inspected. No secret values were printed. .env.example was already modified when the audit started and was left untouched. API key use is server-side in the examined routes; NEXT_PUBLIC Supabase settings are intentionally public/publishable credentials.

7. Tests and executable verification

Command / check	Outcome	What it proves / limits
npm run lint	FAILED; exit 1. 6 errors, 29 warnings across resume, interview, jobs, dashboard, tracker, Topbar.	ESLint scanned configured source. This is a confirmed quality gate failure.
npx tsc --noEmit	PASSED; exit 0.	Current TypeScript sources type-check. Does not exercise runtime behavior, DB calls or provider APIs.
Python compile() of python-parser/main.py	PASSED; syntax OK.	Parser source parses. Does not import/run FastAPI dependencies, test documents or call endpoint.
npm run build	FAILED before app compilation completes; next/font could not fetch Inter from fonts.googleapis.com.	Environment/network-limited. No evidence yet whether the production build otherwise passes.
Automated tests	NOT RUN; none are configured/discovered.	package.json has no test script; no test files/config found. No E2E verification.
Supabase / Gemini / Groq / Adzuna / Remotive / Python runtime	NOT RUN.	No external connections, live DB mutations, payments/messages, uploads or service calls were triggered.

Executed checks were non-destructive. No migrations, deployments or external side effects were invoked. The Next build may refresh ignored .next build artifacts. Git status was checked before and after report generation to ensure source changes were not made.

Existing project documentation reports prior lint and build success, but this audit did not independently verify those historical runs. Current results above supersede them for this checkout/date.

8. Implementation completion estimate

The weighted estimate reflects substantial code presence with incomplete persistence/access paths and no functional verification. It does not treat compilation as feature completion.

Major feature	Weight	Score	Weighted points	Rationale
Auth and profile	15%	0.75	11.25	Sign-up/sign-in/profile/session refresh present; no general route protection; not tested.
Dashboard	10%	0.75	7.50	Live KPI queries and demo feed; live activity and readiness domains are empty.
Resume	20%	0.50	10.00	Parsing, heuristic/optional AI and suggestions present; live analysis persistence absent; upload controls need hardening.
Job discovery	20%	0.75	15.00	Search sources, filters and matching connected; cache context defect and external services unverified.
Application tracker	10%	0.75	7.50	CRUD/status persistence code and demo store; DB behavior not tested.
Interview coach	15%	0.75	11.25	Question bank, evaluator fallback, telemetry and recording code present; DB atomicity and providers untested.
Readiness	10%	0.75	7.50	Task listing/toggle exists; live seed/create and dynamic domain data gaps.
TOTAL	100%		70.00%	sum(weight x score). Scores use 0 / .25 / .50 / .75 / .90 / 1.00 rubric.

Estimated implemented: **70%** (provisional). Estimated feature behavior verified: **0%** of major end-to-end journeys in this audit. External integrations verified: **0%**. Major flows verified: **0 of 7**. Static checks: TypeScript and Python syntax pass; lint fails; production build blocked by font network access. Estimated remaining: **30%** by this weighted model, plus unresolved verification risk.

Confidence: medium-low. The estimate is based on repository-authored feature scope and static code inspection; it is not an acceptance-test score. A provisional score can overstate user-ready behavior because many integrations and persistence paths were not exercised.

Verification coverage must not be interpreted as code coverage: no test suite exists, so automated feature-test coverage is effectively absent. The 0% here means no complete user journey was executed successfully during this audit.

9. Prioritized implementation roadmap

ID / priority	Work, reason and likely files	Depends / effort	Acceptance and verification
P0-1 / P0	Correct match cache context. src/app/api/jobs/match/route.ts	Independent; 0.5-1.5 days	Same job scored for two skill profiles/roles returns distinct correct results; changed content cannot reuse stale result.
P0-2 / P0	Protect AI/file handlers; bound upload size, type and input sizes; add abuse/rate controls. resume analyze route, interview evaluate route, proxy/auth service	Agree guest vs signed-in scope; 2-4 days	Anonymous and oversized requests get intended status; provider not called; legitimate session works; rate limit verified.
P0-3 / P1	Persist newly generated resume analysis and metadata in live mode; avoid claiming success until write completes. resume.service, analyze route, resumes/resume_analyses schema	Auth and schema agreement; 2-4 days	Authenticated analyze -> DB record -> reload retrieves same analysis; write failure surfaces error.
P1-1 / P1	Define protected routes and server-side access checks; maintain intentional demo behavior. src/proxy.ts, layout/pages, route handlers	P0-2 scope decision; 1-3 days	Direct page/API checks pass for anonymous/authenticated/demo states; RLS remains enabled.
P1-2 / P1	Make new-user readiness onboarding credible: seed/create tasks and define domain-score calculation. readiness page/service, migration or server action	Requirements for task catalog; 2-4 days	New account sees actionable tasks, toggle survives refresh, domain score matches documented rule.
P1-3 / P1	Add automated tests for cache, scoring fallback, parser validation, persistence/service behavior and auth. test config, test files, package scripts	P0 fixes; 3-6 days	Repeatable tests run in CI with safe mocks; regressions in top P0 flows fail.
P2-1 / P2	Rerun lint and resolve current 6 errors/29 warnings; fix documented status mismatch. resume route, interview/jobs pages, docs	Can proceed independently; 1-2 days	npm run lint exits 0; docs record exact command/date/environment.
P2-2 / P2	Make build reproducible without live font fetch or verify in network-enabled CI; run full production build. layout/font config, CI	Build environment; 0.5-2 days	npm run build completes in a clean supported environment; route output archived.
P2-3 / P2	Run safe Supabase staging checks: migration, RLS ownership, user lifecycle, writes and reload. SQL migration and service modules	Working staging project/credentials; 2-5 days	Policy checks as different users, CRUD persistence and failure recovery pass; no production data used.
P2-4 / P2	Exercise browser journeys and accessibility/responsive states for each page. app pages/components	P0/P1 stabilised; 3-6 days	E2E evidence for sign-in, upload/analysis/reload, search/match, apply/tracker, interview/save and readiness toggle.

Smallest credible MVP

A single authenticated student can sign in, upload a bounded resume and see a persisted analysis after reload, search and score jobs with isolated correct match context, track an application persistently, complete and save one mock interview, and update at least one readiness task. The MVP is credible only after automated checks plus a safe staging end-to-end run demonstrate those outcomes. Start with P0-1 and P0-2 in parallel; resume persistence depends on auth/data contract; readiness seed can proceed independently once task catalog is agreed; broad E2E validation should follow the fixes.

10. Significant file-by-file inventory

The following inventory covers the key implementation files; references to page/service pairs mean their connected responsibilities were reviewed statically. None was behaviorally tested unless explicitly listed in Section 7.

Repository path	Purpose and current implementation	Connection, incompleteness / verification
src/app/page.tsx; layout.tsx; globals.css	Landing/root route, global shell, fonts/theme and metadata.	Layout mounts Sidebar and Topbar across routes. Font fetch blocks build in this environment; no visual browser check.
src/app/dashboard/page.tsx; src/lib/services/dashboard.service.ts	Dashboard UI and demo/live KPI aggregation.	Connected to applications/session/resume/task tables. Live activity/domain arrays returned empty; service calls not tested.
src/app/jobs/page.tsx; src/lib/services/jobs.service.ts	Search/filter/match UI, apply-to-tracker and save/skill preferences.	Calls search and match APIs; filters client-side. Search fallbacks may return demo data; match errors are swallowed; no browser check.
src/app/tracker/page.tsx; src/lib/services/applications.service.ts	Board/table, add/edit/delete/status movement and service persistence.	Supabase CRUD with user filters; demo memory store. No DB configured or end-to-end verification.
src/app/resume/page.tsx; src/lib/services/resume.service.ts	Resume upload, scoring display, suggestions and skill sync.	Calls analyze API; demo assignment is memory only. Live analysis fetches DB but analysis handler never persists new result.
src/app/interview/page.tsx; src/lib/services/interview.service.ts	Question selection, response capture, camera/mic telemetry, scoring, history and session writes.	Calls evaluate route and Supabase session/turn inserts. Fallback can be deterministic; two-write cleanup is non-atomic.
src/app/readiness/page.tsx; src/lib/services/readiness.service.ts	Milestone list, category/domain UI and completion mutation.	Reads/toggles database rows but no task creation or migration seed; demo data resets with process.
src/app/auth/login/page.tsx; src/app/auth/callback/route.ts; src/lib/services/auth.service.ts	Email sign-in/up, code exchange, user/profile and sign-out.	Supabase operations exist. No credentials; route guard is incomplete; not tested.
src/proxy.ts; src/lib/supabase/proxy.ts	Next.js proxy convention, token refresh and login redirect.	Refresh code is wired. Only login route redirect exists; does not protect other routes.
src/lib/supabase/config.ts; client.ts; server.ts	Credential resolution and Supabase browser/server clients.	Public env aliases supported; dummy placeholder used when unconfigured. No live connection executed.
src/app/api/resume/analyze/route.ts	PDF/DOCX/text extraction, structural heuristic parser, Gemini/Groq scoring and fallback.	Large route with real parser/AI branches. No auth, explicit size ceiling, persistence or fixture tests; lint has 3 any errors.
src/app/api/interview/evaluate/route.ts	Groq/Gemini evaluation and deterministic rubric fallback.	Route connected to UI. No authentication/rate limits; external calls not tested.
src/app/api/jobs/search/route.ts	Adzuna/Remotive/local catalog fallback with source label.	Wired to jobs service. External calls not tested; no Supabase search in this route.
src/app/api/jobs/match/route.ts	Gemini/heuristic job scoring, process cache and request response.	Wired to service; cache omits profile/role/job content inputs; no unit tests.
src/app/api/system/status/route.ts	Reports config flags and optional parser /health probe.	No external DB/model verification; metadata is exposed unauthenticated.
src/components/layout/Sidebar.tsx; Topbar.tsx	Navigation, profile chrome, data mode and system status UI.	Topbar reads status route. No responsive/accessibility/browser review.
src/components/ui/*.tsx	Badge, Button, Card, EmptyState, ErrorBanner, ProgressBar and StatCard primitives.	Reusable UI present. No component tests or visual QA.
src/lib/mock-data.ts; src/types/index.ts	Seeded demo content and shared domain/request response types.	Demo values inform expected user journeys; no tests. API types are not runtime schema validation.
python-parser/main.py; requirements.txt	FastAPI parser with 5 MB cap, PDF/DOCX/TXT extraction and section heuristics.	Optional call from Next route, fallback to native parser. Dependencies not installed/tested as a running service.
supabase/migrations/20261021000000_initial_schema.sql	11 tables, indexes, RLS policies and triggers.	Code suggests per-user policy boundaries; migration was not applied, SQL not executed, no seeded readiness rows.
package.json; package-lock.json; tsconfig.json; eslint.config.mjs; next.config.ts	Dependency/script/build/type/lint configuration.	No test script. TypeScript strict; tsc passed; lint failed; build network-blocked.

11. Limitations, assumptions and appendix

Limitations

- No formal requirements, issue tracker exports, release/deployment records, CI logs, or acceptance-test plan were present in the repository.

- No external credentials were inspected or used. Supabase, AI providers, Adzuna, Remotive and the FastAPI server were not called. Their real-world availability and account configuration are unknown.

- No browser was launched and no UI was visually or accessibly tested. No PDFs/resumes were uploaded; no DB writes were made.

- Build failure is attributable to inability to fetch Google Fonts before app build output, not a proven source compilation defect. A network-enabled CI run is required to resolve that uncertainty.

- The 70% estimate uses a provisional seven-feature scope, equal rubric conventions and weights shown in Section 8. It is an engineering estimate, not a statistical measurement.

Repository and change-safety record

Detected project identity: package name placement-copilot; product label Placement Co-pilot. Framework version next 16.4.2, React 19.3.0 and TypeScript 5.9.3 from manifest/lockfile. Git branch main; HEAD f7387689df781b4ebe4d1c569a60b4a53e78bfe2; latest subject feat: complete placement copilot suite with native resume parsing and AI evaluation (2026-10-29). Initial status showed M .env.example; that change was not edited. Report files are isolated under project-audit-report/.

Useful verification commands

Command	Purpose / safe run condition
npm run lint	Configured ESLint check; currently fails on 6 errors and warnings.
npx tsc --noEmit	Type-check without emitting JavaScript; passed.
npm run build	Production build; requires Google Fonts network access in current configuration, or a verified self-hosted font setup.
python -c compile(Path(...).read_text(), ..., 'exec')	Parser syntax check without creating pycache; passed. Does not run imports/service.
Future: targeted unit/integration/E2E suite	Use mocked providers and isolated staging DB; never exercise production data during initial tests.

Audit conclusion

The codebase is a capable prototype with meaningful integration scaffolding and several real behavior paths. It is not yet demonstrated as an end-to-end persistent application. Fix cache correctness and server-side request boundaries first, make resume results persist in live mode, close the auth/task onboarding gaps, then establish repeatable automated and staging verification before relying on the project's readiness claims.

END OF REPORT